

Skill Week-10

1. To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events

```
GNU nano 8.7.1 src/signal_actions.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void handle_signal(int sig) {
    if (sig == SIGINT)
        printf("\nSIGINT received. Interrupt handled safely.\n");
    else if (sig == SIGTERM)
        printf("SIGTERM received. Program can terminate safely.\n");
}

int main() {
    struct sigaction action;

    action.sa_handler = handle_signal;
    sigemptyset(&action.sa_mask);
    action.sa_flags = 0;

    sigaction(SIGINT, &action, NULL);
    sigaction(SIGTERM, &action, NULL);

    printf("Process PID: %d\n", getpid());
    printf("Signal handlers registered.\n");
    printf("Press Ctrl+C to test SIGINT.\n");

    while (1) {
        printf("Program is running...\n");
        sleep(3);
    }

    return 0;
}
```

Output:

```
akhil@Akhil-laptop: ~$ cd ~/OS_Skill
akhil@Akhil-laptop:~/OS_Skill$ cd Skill-Week-10
akhil@Akhil-laptop:~/OS_Skill/Skill-Week-10$ gcc src/signal_actions.c -o bin/
signal_actions
akhil@Akhil-laptop:~/OS_Skill/Skill-Week-10$ ./bin/signal_actions
Process PID: 1821
Signal handlers registered.
Press Ctrl+C to test SIGINT.

Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
SIGTERM received, Program can terminate safely.
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...

Killed ./bin/signal_actions
```

akhil@Akhil-laptop: ~

×

+

∨

—

□

×

```
akhil@Akhil-laptop:~$ cd ~/OS_Skill
```

```
akhil@Akhil-laptop:~/OS_Skill$ cd Skill-Week-10
```

```
akhil@Akhil-laptop:~/OS_Skill/Skill-Week-10$ gcc src/signal_actions.c -o bin/signal_actions
```

```
akhil@Akhil-laptop:~/OS_Skill/Skill-Week-10$ ./bin/signal_actions
```

```
Process PID: 1821
```

```
Signal handlers registered.
```

```
Press Ctrl+C to test SIGINT.
```

2. To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling

```
GNU nano 8.7.1 src/sigint_forward.c *
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t foreground_pid = -1;

void handle_sigint(int sig) {
    if (foreground_pid > 0) {
        printf("\nShell: Forwarding SIGINT to foreground job.\n");
        kill(foreground_pid, SIGINT);
    } else {
        printf("\nShell: SIGINT received, but no foreground job.\n");
    }
}

int main() {
    struct sigaction action;
    int status;

    action.sa_handler = handle_sigint;
    sigemptyset(&action.sa_mask);
    action.sa_flags = 0;

    sigaction(SIGINT, &action, NULL);

    printf("Shell PID: %d\n", getpid());

    foreground_pid = fork();

    if (foreground_pid < 0) {
        perror("fork");
        return 1;
    }

    if (foreground_pid == 0) {
        printf("Foreground job started. PID: %d\n", getpid());

        signal(SIGINT, SIG_DFL);

        for (int i = 1; i <= 20; i++) {
            printf("Foreground job running... %d\n", i);
            sleep(1);
        }

        return 0;
    }

    printf("Foreground job PID: %d\n", foreground_pid);
    printf("Press Ctrl+C to interrupt the foreground job.\n");

    waitpid(foreground_pid, &status, 0);

    if (WIFSIGNALED(status)) {
        printf("Job terminated by signal %d.\n",
               WTERMSIG(status));
    }

    foreground_pid = -1;

    printf("Shell is still running.\n");

    return 0;
}
```

Output:

```
akhil@Akhil-laptop: ~
akhil@Akhil-laptop:~$ cd ~/OS_Skill
akhil@Akhil-laptop:~/OS_Skill$ cd Skill-Week-10
akhil@Akhil-laptop:~/OS_Skill/Skill-Week-10$ gcc src/sigint_forward.c -o bin/sigint_forward
akhil@Akhil-laptop:~/OS_Skill/Skill-Week-10$ ./bin/sigint_forward
Shell PID: 1902
Foreground job PID: 1903
Press Ctrl+C to interrupt the foreground job.
Foreground job started. PID: 1903
Foreground job running... 1
Foreground job running... 2
Foreground job running... 3
Foreground job running... 4
^C
Shell: Forwarding SIGINT to foreground job.
Shell is still running.
```

3. To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow

```
GNU nano 8.7.1 src/sigtstp_job.c *
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

typedef struct {
    int job_id;
    pid_t pid;
    char state[20];
} Job;

int main() {
    Job job;
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Job started. PID = %d\n", getpid());

        for (int i = 1; i <= 10; i++) {
            printf("Job running... %d\n", i);
            sleep(1);
        }

        printf("Job completed.\n");
        return 0;
    }

    job.job_id = 1;
    job.pid = pid;
    snprintf(job.state, sizeof(job.state), "Running");

    job.pid = pid;
    snprintf(job.state, sizeof(job.state), "Running");

    printf("Job table:\n");
    printf("[%d] PID=%d State=%s\n",
           job.job_id, job.pid, job.state);

    sleep(2);

    printf("\nSending SIGTSTP...\n");
    kill(pid, SIGTSTP);

    waitpid(pid, &status, WUNTRACED);

    if (WIFSTOPPED(status)) {
        snprintf(job.state, sizeof(job.state), "Stopped");

        printf("Job stopped.\n");
        printf("[%d] PID=%d State=%s\n",
               job.job_id, job.pid, job.state);
    }

    sleep(2);

    printf("\nSending SIGCONT...\n");
    kill(pid, SIGCONT);

    snprintf(job.state, sizeof(job.state), "Running");

    printf("[%d] PID=%d State=%s\n",
           job.job_id, job.pid, job.state);

    waitpid(pid, &status, 0);

    if (WIFEXITED(status))
        snprintf(job.state, sizeof(job.state), "Completed");

    printf("\nFinal state:\n");
    printf("[%d] PID=%d State=%s\n",
           job.job_id, job.pid, job.state);

    return 0;
}
```

Output:

```
akhil@Akhil-laptop: ~  
akhil@Akhil-laptop:~/OS_Skill/Skill-Week-10$ gcc src/sigtstp_job.c -o bin/sigtstp_job  
akhil@Akhil-laptop:~/OS_Skill/Skill-Week-10$ ./bin/sigtstp_job  
Job table:  
[1] PID=1979 State=Running  
Job started. PID = 1979  
Job running... 1  
Job running... 2  
  
Sending SIGTSTP...  
Job stopped.  
[1] PID=1979 State=Stopped  
  
Sending SIGCONT...  
[1] PID=1979 State=Running  
Job running... 3  
Job running... 4  
Job running... 5  
Job running... 6  
Job running... 7  
Job running... 8  
Job running... 9  
Job running... 10  
Job completed.  
  
Final state:  
[1] PID=1979 State=Completed
```

4. To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior

```
GNU nano 8.7.1 src/process_group_terminal.c *
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <termios.h>

int main() {
    pid_t shell_pgid;
    pid_t child_pid;
    int status;

    shell_pgid = getpgid();

    printf("Shell PID : %d\n", getpid());
    printf("Shell PGID: %d\n", shell_pgid);

    child_pid = fork();

    if (child_pid < 0) {
        perror("fork");
        return 1;
    }

    if (child_pid == 0) {
        setpgid(0, 0);

        printf("\nChild PID : %d\n", getpid());
        printf("Child PGID: %d\n", getpgid());

        sleep(3);

        printf("Child: Foreground job running.\n");

        return 0;
    }

    setpgid(child_pid, child_pid);
    printf("\nCreated process group.\n");

    printf("Child process group ID: %d\n", child_pid);
    sleep(1);
    printf("\nTransferring terminal to child group...\n");
    tcsetpgrp(STDIN_FILENO, child_pid);

    printf("Terminal control transferred.\n");
    waitpid(child_pid, &status, 0);

    printf("\nChild completed.\n");
    printf("Restoring terminal to shell group...\n");
    tcsetpgrp(STDIN_FILENO, shell_pgid);

    printf("Terminal control restored to shell.\n");

    return 0;
}
```

Output:

```
akhil@Akhil-laptop:~/OS_Skill/Skill-Week-10$ nano src/process_group_terminal
.c
akhil@Akhil-laptop:~/OS_Skill/Skill-Week-10$ gcc src/process_group_terminal.
c -o bin/process_group_terminal
akhil@Akhil-laptop:~/OS_Skill/Skill-Week-10$ ./bin/process_group_terminal
Shell PID : 2023
Shell PGID: 2023

Created process group.
Child process group ID: 2024

Child PID : 2024
Child PGID: 2024

Transferring terminal to child group...
Terminal control transferred.
Child: Foreground job running.

Child completed.
Restoring terminal to shell group...

[1]+  Stopped                  ./bin/process_group_terminal
```